

Protocolo de Code Review: prevención de inyección SQL en PHP y MySQL

1. Objetivo del protocolo

Comprobar que ningún dato externo (formularios, URL, cookies, cabeceras, archivos subidos o datos leídos de la propia base de datos) pasa a formar parte del **texto** de una consulta SQL. Los datos deben viajar siempre **por separado**, como parámetros de una consulta preparada.

Por qué importa: una inyección SQL permite a un atacante leer, modificar o borrar datos, o saltarse el login.

Alcance: todo archivo PHP del PR que ejecute SQL (controladores, modelos, repositorios, scripts de utilidad).

Regla de oro: si un valor viene de fuera, nunca se concatena en la consulta. Se envía con un marcador (`?` o `:nombre`).

2. Pasos principales de la revisión

Paso 1. Localizar todas las consultas SQL del PR

Busca estas funciones y métodos:

- `mysqli_query()`, `$mysqli->query()`, `$pdo->query()`, `$pdo->exec()`
- `mysqli_multi_query()`
- Cadenas que contengan `SELECT`, `INSERT`, `UPDATE`, `DELETE` o `WHERE`

Motivo: no se puede revisar lo que no se ha localizado. Cada punto donde se ejecuta SQL es una revisión independiente.

Paso 2. Identificar el origen de cada variable de la consulta

Sigue cada variable hacia atrás. Son **no confiables**: `$_GET`, `$_POST`, `$_REQUEST`, `$_COOKIE`, `$_SERVER` (cabeceras), `$_FILES`, el cuerpo JSON (`php://input`) y los datos leídos de la BD.

Motivo: un dato guardado previamente en la base de datos pudo ser introducido por un atacante (inyección de segundo orden).

Paso 3. Comprobar cómo se construye la consulta

Es la revisión principal. Si hay datos externos dentro de la consulta mediante `.` (concatenación) o interpolación `"...$variable..."`, es un hallazgo.

Motivo: la concatenación mezcla datos con instrucciones SQL, y el motor no puede distinguirlos.

Paso 4. Verificar que se usan consultas preparadas

Debe haber un texto de consulta fijo con marcadores y los valores enviados aparte con `execute()` o `bind_param()`.

Motivo: el motor recibe la estructura primero y los valores después, así que un valor nunca puede alterar la consulta.

Paso 5. Revisar nombres dinámicos (**ORDER BY**, tablas, columnas)

Los marcadores `?` solo sirven para valores, no para identificadores. Si el nombre de columna viene del cliente, debe validarse contra una **lista blanca**.

Motivo: aquí es donde más se cae en la tentación de concatenar.

Paso 6. Revisar **LIKE**, **IN** y **LIMIT**

- **LIKE**: el comodín `%` va dentro del valor, no en la consulta.
- **IN**: un marcador por cada elemento.
- **LIMIT**: convertir a entero con `(int)` y fijar un máximo.

Motivo: son casos donde es habitual abandonar los parámetros por comodidad.

Paso 7. Revisar configuración y manejo de errores

- Conexión PDO con `ATTR_EMULATE_PREPARES => false` y `ERRMODE_EXCEPTION`.
- Los errores de MySQL no se muestran al usuario (`display_errors` desactivado en producción; mensajes genéricos).
- El usuario de BD de la aplicación no es `root` y solo tiene los permisos necesarios.

Motivo: limita el daño si algo falla y evita filtrar información del esquema.

Paso 8. Comprobar los tests

Debe haber pruebas con entradas como `' OR '1'='1, '; DROP TABLE users; --` y nombres legítimos con comilla (`0'Brien`).

Motivo: garantiza que la corrección funciona y que no se rompe en el futuro.

3. Ejemplos en PHP: código vulnerable y versión segura

Ejemplo 1: consulta con un parámetro (PDO)

php

//  VULNERABLE

\$email = \$_POST['email'];

```
$sql = "SELECT * FROM users WHERE email = '$email'";  
$result = $pdo->query($sql);
```

Si el usuario envía ' OR '1'='1', la consulta devuelve todos los usuarios.

```
php  
//  SEGURO (PDO)  
$stmt = $pdo->prepare("SELECT * FROM users WHERE email = :email");  
$stmt->execute([':email' => $_POST['email']]);  
$user = $stmt->fetch();
```

Ejemplo 2: MySQLi

```
php  
//  VULNERABLE  
$id = $_GET['id'];  
$result = $mysqli->query("SELECT * FROM products WHERE id = " . $id);
```

Aunque `id` sea numérico, el usuario puede enviar `1 OR 1=1` o algo peor.

```
php  
//  SEGURO (MySQLi)  
$stmt = $mysqli->prepare("SELECT * FROM products WHERE id = ?");  
$stmt->bind_param("i", $_GET['id']); // "i" = entero  
$stmt->execute();  
$result = $stmt->get_result();
```

Ejemplo 3: INSERT con varios datos

```
php  
//  VULNERABLE  
$sql = "INSERT INTO comments (user_id, text) VALUES ($userId, '$text')";  
$pdo->exec($sql);  
  
//  SEGURO  
$stmt = $pdo->prepare("INSERT INTO comments (user_id, text) VALUES (?, ?)");  
$stmt->execute([$userId, $text]);
```

Ejemplo 4: ORDER BY dinámico (lista blanca)

```
php  
//  VULNERABLE  
$sql = "SELECT * FROM products ORDER BY " . $_GET['sort'];  
  
//  SEGURO  
$permitidas = ['nombre' => 'name', 'precio' => 'price'];  
$columna = $permitidas[$_GET['sort']] ?? 'name';  
$orden = (($_GET['dir'] ?? '') === 'desc') ? 'DESC' : 'ASC';  
  
$stmt = $pdo->query("SELECT * FROM products ORDER BY $columna $orden");
```

Aquí la interpolación es aceptable porque `$columna` y `$orden` solo pueden tomar valores definidos por el desarrollador.

Ejemplo 5: **LIKE** e **IN**

```
php
//  LIKE seguro
$stmt = $pdo->prepare("SELECT * FROM products WHERE name LIKE ?");
$stmt->execute(['%' . $texto . '%']);

//  IN seguro
$ids = [1, 2, 3];
$marcadores = implode(',', array_fill(0, count($ids), '?'));
$stmt = $pdo->prepare("SELECT * FROM users WHERE id IN ($marcadores)");
$stmt->execute($ids);
```

Ejemplo 6: **LIMIT**

```
php
//  SEGURO
$limit = min((int)($_GET['limit'] ?? 10), 100);
$stmt = $pdo->query("SELECT * FROM products LIMIT $limit");
```

Conexión PDO recomendada

```
php
$pdo = new PDO(
    'mysql:host=localhost;dbname=app;charset=utf8mb4',
    $user, $pass,
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]
);
```

4. Errores frecuentes que debe detectar el desarrollador

Error	Por qué es un problema	Cómo corregirlo
Concatenar o interpolar variables en el SQL ("... '\$x' ", "... " . \$x)	El dato pasa a ser parte de la consulta	Consulta preparada con marcadores
Usar <code>mysqli_real_escape_string()</code> o <code>addslashes()</code> como única protección	Es frágil, depende del charset y se olvida fácilmente (y no protege valores numéricos sin comillas)	Consultas preparadas

Usar <code>prepare()</code> pero con la variable ya concatenada dentro del texto	Parece seguro, pero no lo es: la consulta sigue siendo dinámica	Poner <code>?</code> y pasar el valor en <code>execute()</code>
Crear que "es un número" sin comprobarlo (<code>WHERE id = \$id</code>)	Sin comillas, no hay nada que escapar y el atacante inyecta SQL directamente	Usar <code>bind_param("i", ...)</code> o <code>(int)</code>
<code>ORDER BY</code> , tablas o columnas tomadas del cliente sin lista blanca	Los marcadores no sirven para identificadores	Lista blanca
Confiar en datos de la base de datos	Pudieron ser insertados por un atacante	Parametrizar también esas consultas
Construir <code>UPDATE/INSERT</code> con las claves del array <code>\$_POST</code> como nombres de columna	El atacante elige qué columnas modificar	Lista blanca de campos permitidos
Mostrar <code>\$mysqli->error</code> o excepciones PDO al usuario	Revela estructura de la BD	Mensaje genérico y log interno
Conectar con <code>root</code>	Una inyección tendría control total	Usuario con permisos mínimos
Usar <code>mysqli_multi_query()</code> sin necesidad	Permite ejecutar varias sentencias encadenadas	Usar <code>execute()</code> normal
Validar solo en el frontend (JavaScript)	El atacante puede saltarse el navegador	Validar y parametrizar en el servidor

5. Checklist final: ¿supera el código la revisión?

El PR **solo se aprueba** si todas las casillas obligatorias están marcadas.

Obligatorias (bloqueantes)

- He localizado todas las consultas SQL del PR.
- He identificado el origen de cada variable que entra en ellas.
- Ninguna consulta concatena ni interpola datos externos.
- Todas las consultas con datos externos usan consultas preparadas (`prepare + execute/bind_param`).
- `ORDER BY`, tablas y columnas dinámicas usan lista blanca.
- Las claves de `$_POST/$_GET` no se usan como nombres de columna.

- No se usa `mysqli_real_escape_string()` ni `addslashes()` como defensa única.

Importantes (corregir antes de aprobar o justificar)

- `LIKE`, `IN` y `LIMIT` están bien tratados.
- Los errores de MySQL no se muestran al usuario.
- PDO usa `ERRMODE_EXCEPTION` y `ATTR_EMULATE_PREPARES => false`.

Recomendadas (pueden ir en otro PR)

- El usuario de BD tiene permisos mínimos (no `root`).
- Existen tests con entradas maliciosas y con caracteres especiales legítimos.

Cómo dejar el comentario en el PR: indica el archivo y la línea, explica el riesgo en una frase y propone la corrección.

● **Bloqueante:** en `UserModel.php`, línea 27, `$email` se concatena en la consulta. Un atacante puede alterar la consulta. Sugerencia: usar `$pdo->prepare()` con `:email` y pasar el valor en `execute()`.

Ante la duda, **pregunta o escala** a un compañero con más experiencia en seguridad antes de aprobar.